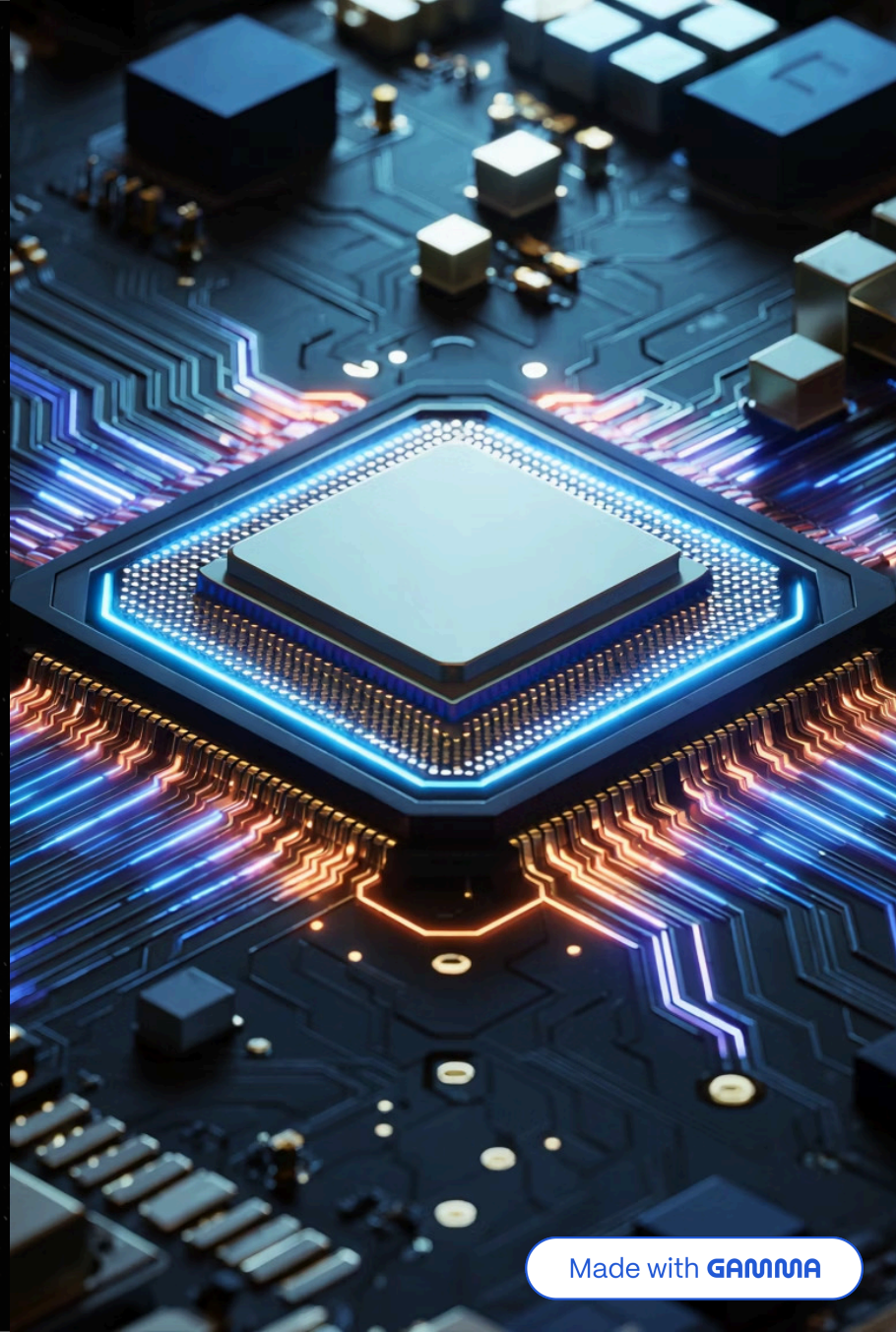
Operating Systems Checkpoint

A comprehensive simulation of four fundamental operating system concepts: **CPU Scheduling**, **Process Synchronization**, **Memory Management**, and **Disk Scheduling**. All simulations were implemented using JavaScript and executed with Node.js.

OPERATING SYSTEMS | SYSTEMS PROGRAMMING | ALGORITHMS



Project Overview

This project simulates several important concepts used by modern operating systems. The objective is to understand how an operating system schedules processes on the CPU, protects shared resources from race conditions, manages memory using page replacement algorithms, and optimizes disk access through scheduling techniques. The project is divided into four independent parts, each exploring a distinct OS mechanism.

Part 1

Process Scheduling — Decides which process should use the CPU and for how long. Algorithms: FCFS and Round Robin.

Part 2

Process Synchronization — Protects shared resources from race conditions using mutex locks to ensure data consistency.

Part 3

Memory Management — Manages limited memory efficiently using page replacement algorithms: FIFO and LRU.

Part 4

Disk Scheduling — Optimizes disk access by reducing head movement through scheduling techniques: FCFS and SSTF.

ⓘ All four simulations were implemented using **JavaScript** and executed with **Node.js**, providing practical experience with core operating system mechanisms.

PART 1

Process Scheduling

The goal of process scheduling is to decide **which process should use the CPU and for how long**. An effective scheduler maximizes CPU utilization, minimizes waiting time, and ensures fair access for all processes. Three processes were used in this simulation:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1

Two algorithms were implemented and compared: **FCFS (First Come First Served)** and **Round Robin** with a time quantum of 2 units. Each algorithm was evaluated based on average waiting time and responsiveness.

PART 1 — ALGORITHMS

FCFS vs. Round Robin

FCFS — First Come First Served

FCFS executes processes strictly according to their arrival order. Execution order: **P1 → P2 → P3**. This non-preemptive algorithm is very simple to implement with low overhead, but short processes may wait a long time behind longer ones, resulting in poor response time for interactive systems.

Round Robin — Quantum = 2

Round Robin gives each process a fixed amount of CPU time called a **time quantum**. When the quantum expires, the process is preempted and moved to the back of the queue. This approach provides fair CPU sharing, better responsiveness, and is commonly used in multitasking systems. The trade-off is more context switching and slightly higher overhead.

3.33	3.33	2
FCFS Avg. Wait	RR Avg. Wait	Time Quantum
Average waiting time for FCFS scheduling	Average waiting time for Round Robin	Fixed CPU time units per RR slice

Although both algorithms produced the same average waiting time in this example, **Round Robin is considered more responsive** because every process gets CPU access regularly, preventing any single process from monopolizing the CPU.

PART 2

Process Synchronization

When multiple processes access the same resource, **data corruption may occur** if synchronization is not used. This part demonstrates the classic race condition problem and its solution using mutual exclusion locks.

The Scenario

Two processes increment a shared variable called "**counter**". P1 increments the counter 100 times, and P2 also increments the counter 100 times. The expected final value is **200**.

The Problem: Race Condition

Without synchronization, both processes may read and write the same value simultaneously. For example, if Counter = 5, both P1 and P2 may read 5, increment to 6, and write 6 — resulting in a final value of 6 instead of the expected 7. **One update is lost.** This situation is called a **Race Condition**.

The Solution: Mutex

A **Mutex (Mutual Exclusion Lock)** is used to ensure that only one process can enter the critical section at a time. This prevents race conditions, preserves data consistency, and guarantees correct results. With the mutex in place, the final counter value is correctly **200**.

Final Result: Counter = 200 (correct). Synchronization is essential whenever multiple processes share data.

PART 3

Memory Management

The goal is to manage **limited memory efficiently** using page replacement algorithms. When a process needs a page that is not in memory, a **page fault** occurs, and the OS must decide which existing page to evict to make room. The simulation uses a reference string of **1, 2, 3, 2, 4, 1, 5** with **3 available frames**.

FIFO — First In First Out

FIFO removes the **oldest page** currently in memory, regardless of how frequently it has been used. It is easy to implement with low overhead, but it may remove frequently used pages, leading to unnecessary page faults. Result: **6 page faults**.

LRU — Least Recently Used

LRU removes the page that has **not been used for the longest time**. This better reflects actual usage patterns and usually produces fewer page faults than FIFO. However, it requires more complex implementation to track usage history. Result: **6 page faults**.

FIFO Page Faults

6 out of 7 references caused a page fault

LRU Page Faults

6 out of 7 references caused a page fault

For this specific reference string, both algorithms performed equally with 6 page faults each. However, in real systems, **LRU generally performs better** because it adapts to actual usage patterns rather than simply evicting the oldest page.

PART 4

Disk Scheduling

Disk scheduling determines the **order in which disk requests are served** to reduce head movement and improve performance. Excessive head movement increases latency and reduces throughput. The simulation starts with the disk head at position **53** and must serve the following pending requests: **98, 183, 37, 122, 14, 124, 65, 67**.

FCFS — First Come First Served

Serves requests strictly in the order they arrive. Simple to implement and fair in request ordering, but results in large head movement and lower efficiency. **Total head movement: 640 tracks.**

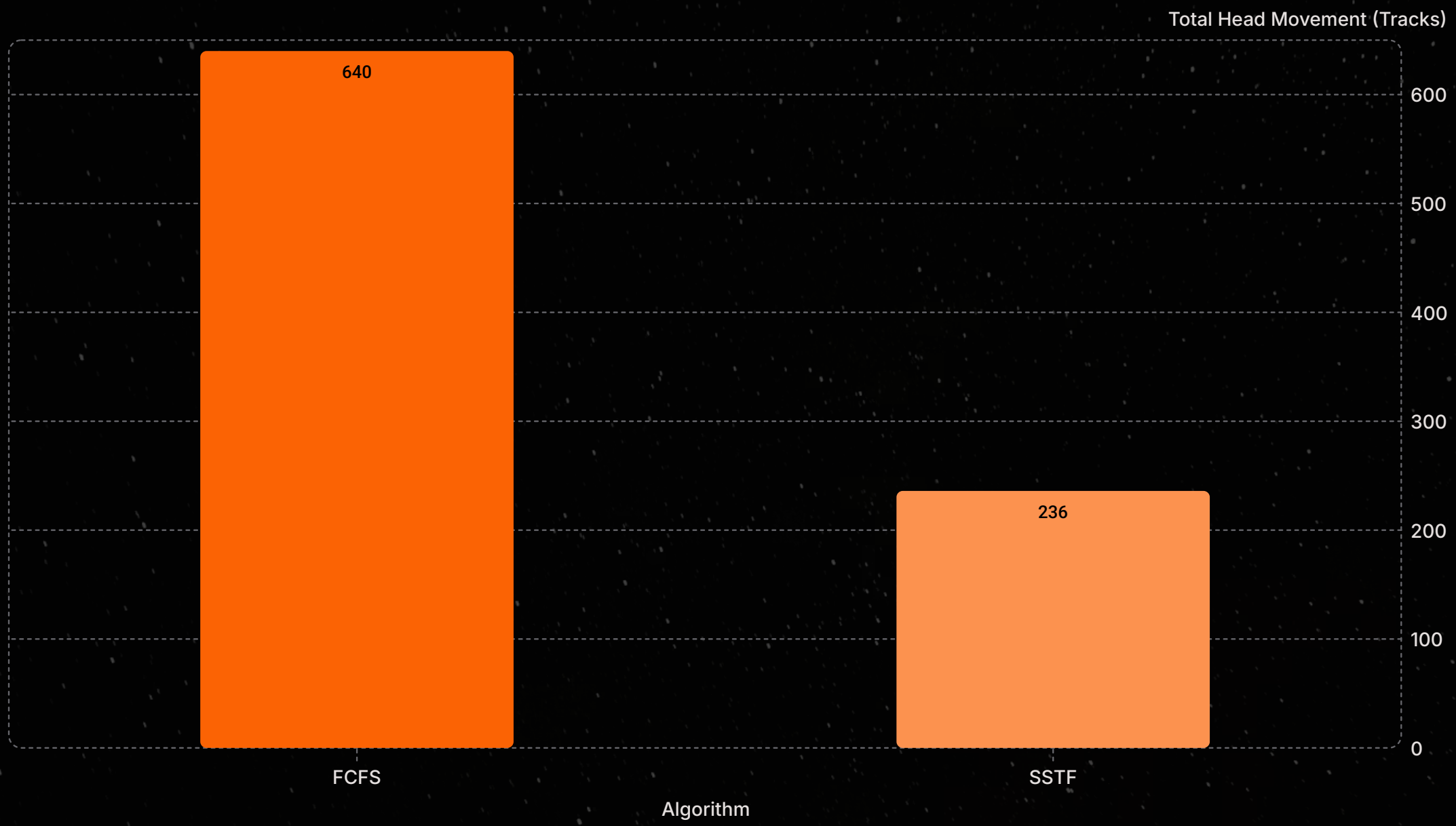
SSTF — Shortest Seek Time First

Always selects the **nearest request** to the current head position. Reduces seek time and improves performance, though some requests may wait longer (potential starvation). **Total head movement: 236 tracks.**

PART 4 — RESULTS

Disk Scheduling: Head Movement Comparison

The difference in efficiency between FCFS and SSTF is dramatic. SSTF reduces total head movement by **63%** compared to FCFS, demonstrating the significant performance gains achievable through intelligent disk scheduling.



FCFS Analysis

FCFS serves requests in the order they arrive without any optimization. The head may travel back and forth across the disk extensively, resulting in **640 tracks** of total movement. While simple and fair, this approach is inefficient for workloads with scattered request patterns.

SSTF Analysis

SSTF always selects the nearest request, minimizing seek distance at each step. This greedy approach reduces total head movement to just **236 tracks** — a **63% reduction** compared to FCFS. SSTF significantly reduces disk head movement and therefore provides better efficiency than FCFS.

Key Results Summary

The following table summarizes the key quantitative results from all four parts of the simulation, providing a quick reference for comparing algorithm performance across each operating system concept.

Part	Algorithm	Metric	Result
1 — CPU Scheduling	FCFS	Avg. Waiting Time	3.33
1 — CPU Scheduling	Round Robin (Q=2)	Avg. Waiting Time	3.33
2 — Synchronization	Without Mutex	Final Counter Value	6 (expected 7)
2 — Synchronization	With Mutex	Final Counter Value	200 (correct)
3 — Memory Mgmt.	FIFO	Page Faults	6
3 — Memory Mgmt.	LRU	Page Faults	6
4 — Disk Scheduling	FCFS	Head Movement	640 tracks
4 — Disk Scheduling	SSTF	Head Movement	236 tracks

63%

Head Movement Reduction

SSTF vs. FCFS in disk scheduling

200

Correct Counter Value

Achieved with mutex synchronization

3.33

Avg. Wait Time

Both FCFS and Round Robin (this example)

Final Conclusion

This project demonstrated four fundamental operating system concepts through hands-on JavaScript/Node.js simulations. Each part illustrated how different algorithms affect system performance, data integrity, and resource efficiency.

1 CPU Scheduling

FCFS and Round Robin were simulated using processes P1, P2, and P3. Both produced an average waiting time of 3.33, but Round Robin showed better responsiveness because every process gets CPU access regularly through time-slicing with a quantum of 2 units.

2 Process Synchronization

Mutex locks prevented race conditions when two processes incremented a shared counter 100 times each. Without synchronization, one update was lost (final value 6 instead of 7); with the mutex, the correct final value of 200 was achieved, demonstrating that synchronization is essential whenever multiple processes share data.

3 Memory Management

FIFO and LRU page replacement algorithms were compared using reference string 1, 2, 3, 2, 4, 1, 5 with 3 frames. Both generated 6 page faults in this example, though LRU generally performs better in real systems because it reflects actual usage patterns.

4 Disk Scheduling

FCFS and SSTF were simulated with initial head position 53 and requests 98, 183, 37, 122, 14, 124, 65, 67. SSTF reduced head movement from 640 to 236 tracks — a 63% improvement — demonstrating significantly better efficiency than FCFS.

Overall, the project provided practical experience with core operating system mechanisms and showed how different algorithms affect system performance and efficiency.